Experiment 21: Kth Missing Positive Number

Aim: To find the kth positive integer missing from a strictly increasing sorted array.

Algorithm:

1. Read the sorted array arr and the integer k.
2. Start with the first positive integer and scan forward.
3. If the current positive integer is not equal to the next array element, count it as missing.
4. Return the missing integer when the missing count reaches k.
5. Otherwise, advance the array index when a match is found and continue.

Code (Python):

```
def find_kth_positive(arr, k):
    missing = 0
    current = 1
    i = 0
    while True:
        if i < len(arr) and arr[i] == current:
            i += 1
        else:
            missing += 1
            if missing == k:
                return current
        current += 1


print(find_kth_positive([2, 3, 4, 7, 11], 5))
print(find_kth_positive([1, 2, 3, 4], 2))
```

Input:

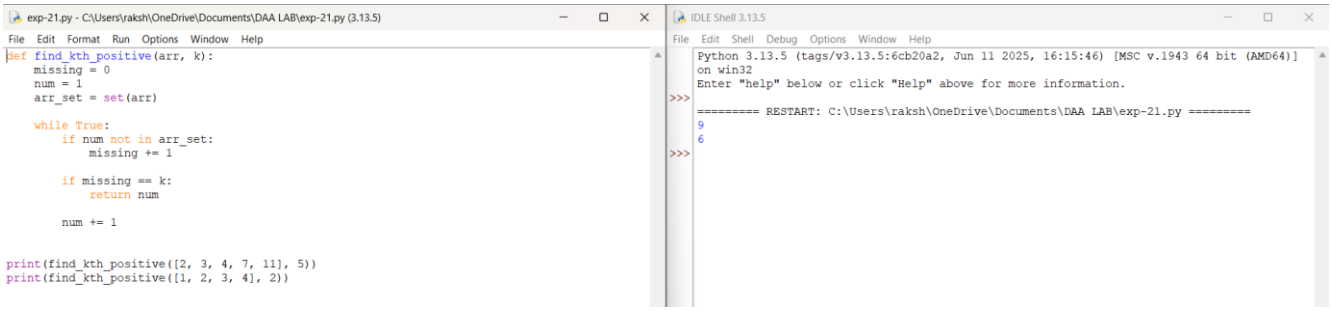
arr = [2, 3, 4, 7, 11], k = 5

arr = [1, 2, 3, 4], k = 2

Output:

9

6



Time Complexity: O(n + k)

Space Complexity: O(1)

Result: The program successfully returned the kth missing positive integer for both test cases.

Experiment 22: Find a Peak Element Using Binary Search

Aim: To find the index of any peak element in an integer array in O(log n) time.

Algorithm:

1. Initialize left to 0 and right to the last index.
2. While left is less than right, calculate the middle index.
3. If nums[mid] is less than nums[mid + 1], a peak lies to the right; set left to mid + 1.
4. Otherwise, set right to mid.
5. When left equals right, return that index.

Code (Python):

```
def find_peak_element(nums):
    left, right = 0, len(nums) - 1
    while left < right:
        mid = (left + right) // 2
        if nums[mid] < nums[mid + 1]:
            left = mid + 1
        else:
            right = mid
    return left


print(find_peak_element([1, 2, 3, 1]))
print(find_peak_element([1, 2, 1, 3, 5, 6, 4]))
```

Input:

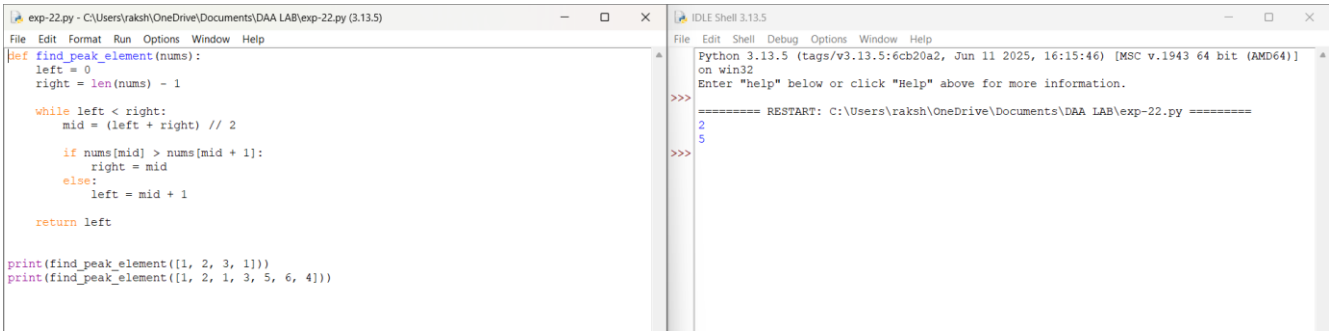
nums = [1, 2, 3, 1]

nums = [1, 2, 1, 3, 5, 6, 4]

Output:

2

5



Time Complexity: O(log n)

Space Complexity: O(1)

Result: The binary search algorithm returned a valid peak index for each input.

Experiment 23: First Occurrence of a Substring

Aim: To return the index of the first occurrence of needle in haystack, or -1 when it is not present.

Algorithm:

1. Read haystack and needle.
2. If needle is empty, return 0.
3. Check each possible starting index in haystack where needle could fit.
4. Compare the substring at that index with needle.
5. Return the first matching index; if no match is found, return -1.

Code (Python):

```
def str_str(haystack, needle):
    if needle == "":
        return 0
    for i in range(len(haystack) - len(needle) + 1):
        if haystack[i:i + len(needle)] == needle:
            return i
    return -1


print(str_str("sadbutsad", "sad"))
print(str_str("leetcode", "leeto"))
```

Input:

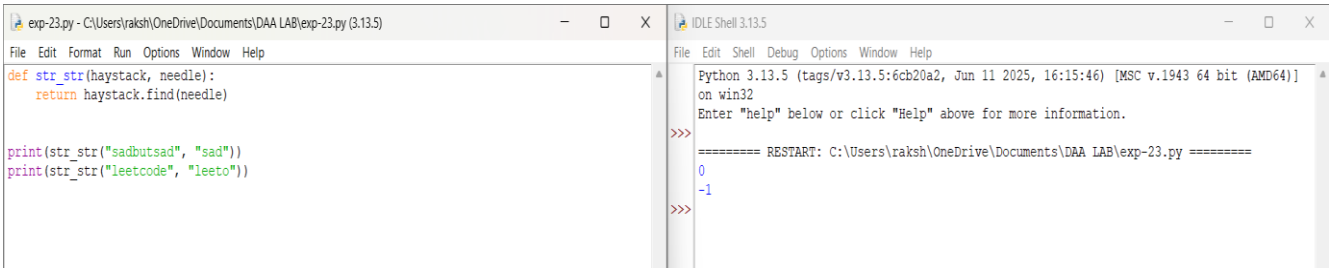
haystack = "sadbutsad", needle = "sad"

haystack = "leetcode", needle = "leeto"

Output:

0

-1



Time Complexity: O(n × m)

Space Complexity: O(1)

Result: The program found the first occurrence when present and returned -1 when the substring was absent.

Experiment 24: Strings That Are Substrings of Other Words

Aim: To return every word that appears as a contiguous substring of at least one other word in the given list.

Algorithm:

1. Read the list of words.
2. For each word, compare it with every different word in the list.
3. If another word contains it, add it to the answer and stop checking that word.
4. Return the list of matching words.

Code (Python):

```
def string_matching(words):
    answer = []
    for i, word in enumerate(words):
        for j, other in enumerate(words):
            if i != j and word in other:
                answer.append(word)
                break
    return answer
print(string_matching(["mass", "as", "hero", "superhero"]))
print(string_matching(["leetcode", "et", "code"]))
print(string_matching(["blue", "green", "bu"]))
```

Input:

["mass", "as", "hero", "superhero"]

["leetcode", "et", "code"]

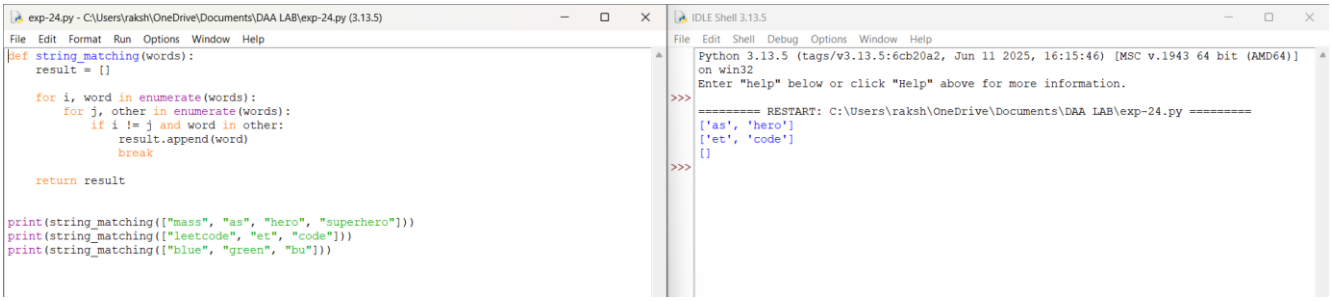
["blue", "green", "bu"]

Output:

["as", "hero"]

["et", "code"]

[]



Time Complexity: $O(n^2 \times L)$

Space Complexity: $O(n)$

Result: The program correctly identified words that occur as substrings of other list entries.

Experiment 25: Closest Pair of Points Using Brute Force

Aim: To find the pair of 2D points with the minimum Euclidean distance using the brute-force method.

Algorithm:

1. Read the list of points.
2. Set the minimum distance to infinity.
3. Compare every pair of distinct points.
4. Calculate the Euclidean distance for each pair and update the closest pair when a smaller distance is found.
5. Display the closest pair and the minimum distance.

Code (Python):

```
from math import hypot
def closest_pair(points):
    best_pair = None
    best_distance = float("inf")
    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            distance = hypot(points[i][0] - points[j][0],
                             points[i][1] - points[j][1])
            if distance < best_distance:
                best_distance = distance
                best_pair = (points[i], points[j])
    return best_pair, best_distance

points = [(1, 2), (4, 5), (7, 8), (3, 1)]
pair, distance = closest_pair(points)
print("Closest pair:", pair[0], "-", pair[1])
print("Minimum distance:", distance)
```

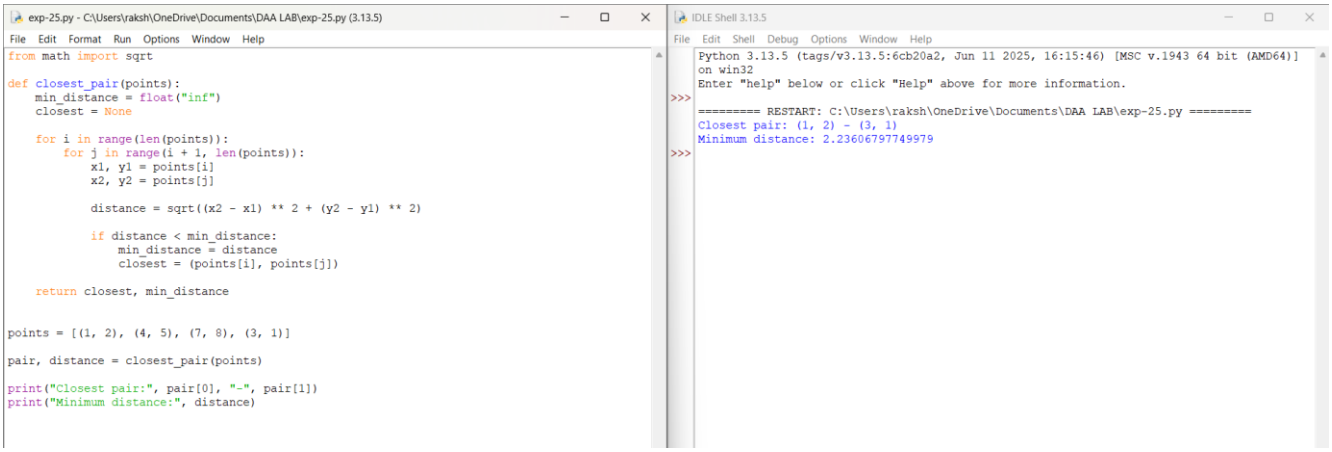
Input:

```
points = [(1, 2), (4, 5), (7, 8), (3, 1)]
```

Output:

Closest pair: (1, 2) - (3, 1)

Minimum distance: 2.23606797749979



Time Complexity: $O(n^2)$

Space Complexity: $O(1)$

Result: The brute-force method found the closest pair by checking all possible point pairs.

Experiment 26: Brute-Force Convex Hull with Collinear Points

Aim: To determine the boundary points of a set of 2D points using a brute-force convex-hull method and handle collinear points without treating interior points on a boundary line as separate hull vertices.

Algorithm:

1. Read the set of points and initialize an empty hull set.
2. Consider every pair of distinct points as a candidate boundary line.
3. Compute the cross-product sign for every other point relative to the candidate line.
4. If all other points lie on one side or on the line, the pair lies on a supporting boundary line.
5. Add the extreme endpoints of each supporting line to the hull set; this avoids adding points that lie strictly between two collinear boundary points.
6. Sort the hull vertices around their centroid in counter-clockwise order and display them.

Code (Python):

```
from math import atan2


def convex_hull(points):
    if len(points) <= 1:
        return points[:]
    hull = set()
    n = len(points)
    for i in range(n):
        for j in range(i + 1, n):
            x1, y1 = points[i]
            x2, y2 = points[j]
            signs = []
            for k in range(n):
                if k == i or k == j:
                    continue
                x, y = points[k]
                cross = (x2 - x1) * (y - y1) - (y2 - y1) * (x - x1)
                if cross != 0:
                    signs.append(1 if cross > 0 else -1)
            if not (1 in signs and -1 in signs):
```

```
                hull.add(points[i])
                hull.add(points[j])
        cx = sum(p[0] for p in hull) / len(hull)
        cy = sum(p[1] for p in hull) / len(hull)
        return sorted(hull, key=lambda p: atan2(p[1] - cy, p[0] - cx))

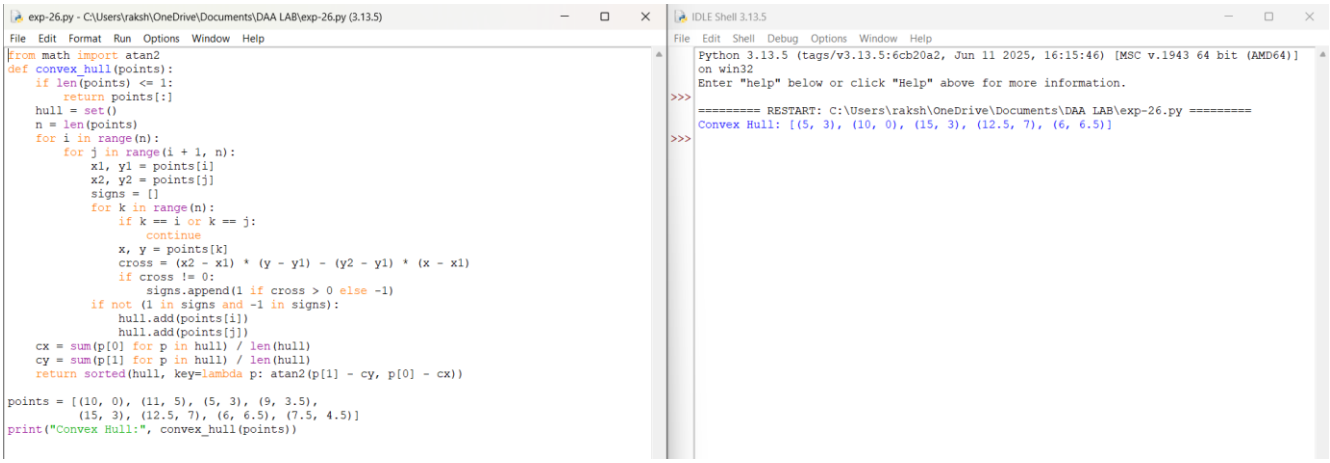

    points = [(10, 0), (11, 5), (5, 3), (9, 3.5),
              (15, 3), (12.5, 7), (6, 6.5), (7.5, 4.5)]
    print("Convex Hull:", convex_hull(points))
```

Input: [(10, 0), (11, 5), (5, 3), (9, 3.5),
(15, 3), (12.5, 7), (6, 6.5), (7.5, 4.5)]

Output:

Convex Hull (counter-clockwise, starting point may vary):

[(5, 3), (10, 0), (15, 3), (12.5, 7), (6, 6.5)]



Time Complexity: $O(n^3)$

Space Complexity: O(n)

Result: The brute-force approach identified the outer boundary. Collinear boundary points are handled by retaining extreme endpoints rather than all points on the same line.

Experiment 27: Convex Hull of 2D Points

Aim: To find the points forming the convex hull of a set of 2D points in counter-clockwise order using a brute-force approach.

Algorithm:

1. Read the list of 2D points.
2. Examine every pair of points as a candidate hull edge.
3. Use the cross product to determine whether all other points lie on the same side of the candidate edge.
4. Collect the endpoints of valid supporting edges.
5. Sort the collected hull points around their centroid and display them in counter-clockwise order.

Code (Python):

```
from math import atan2


def convex_hull(points):
    if len(points) <= 1:
        return points[:]
    hull = set()
    n = len(points)
    for i in range(n):
        for j in range(i + 1, n):
            x1, y1 = points[i]
            x2, y2 = points[j]
            signs = []
            for k in range(n):
                if k == i or k == j:
                    continue
                x, y = points[k]
                cross = (x2 - x1) * (y - y1) - (y2 - y1) * (x - x1)
                if cross != 0:
                    signs.append(1 if cross > 0 else -1)
            if not (1 in signs and -1 in signs):
                hull.add(points[i])
```

```
                hull.add(points[j])
        cx = sum(p[0] for p in hull) / len(hull)
        cy = sum(p[1] for p in hull) / len(hull)
        return sorted(hull, key=lambda p: atan2(p[1] - cy, p[0] - cx))

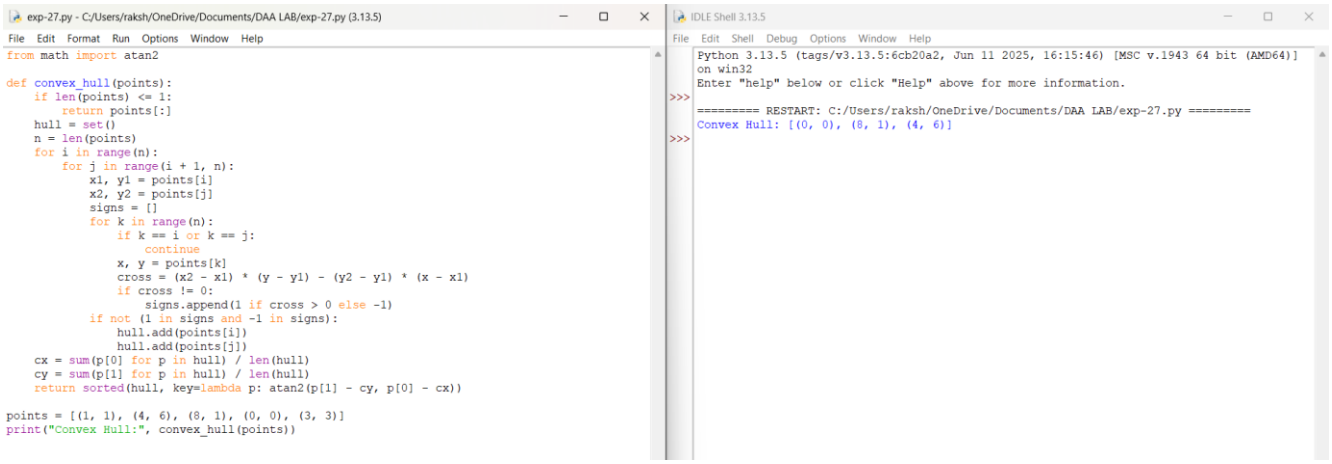

    points = [(1, 1), (4, 6), (8, 1), (0, 0), (3, 3)]
    print("Convex Hull:", convex_hull(points))
```

Input:

points = [(1, 1), (4, 6), (8, 1), (0, 0), (3, 3)]

Output:

Convex Hull: [(0, 0), (8, 1), (4, 6)]



Time Complexity: $O(n^3)$.

Space Complexity: O(n)

Result: The program returned the outermost points in counter-clockwise order. Interior points were excluded from the convex hull.

Experiment 28: Travelling Salesperson Problem Using Exhaustive Search

Aim: To find the shortest round-trip route through a set of cities by checking every permutation of the cities other than the fixed starting city.

Algorithm:

1. Define a function to calculate Euclidean distance between two cities.
2. Fix the first city as the starting city.
3. Generate every permutation of the remaining cities.
4. For each route, calculate the distance of all consecutive legs and the return leg to the start.
5. Keep the route with the smallest total distance and display its path and distance.

Code (Python):

```
from itertools import permutations
from math import hypot

def distance(city1, city2):
    return hypot(city1[0] - city2[0], city1[1] - city2[1])

def tsp(cities):
    if len(cities) <= 1:
        return 0, cities + cities
    start = cities[0]
    best_distance = float("inf")
    best_path = None
    for order in permutations(cities[1:]):
        path = [start] + list(order) + [start]
        total = sum(distance(path[i], path[i + 1])
                    for i in range(len(path) - 1))
        if total < best_distance:
            best_distance = total
            best_path = path
    return best_distance, best_path

tests = [
```

```
    [(1, 2), (4, 5), (7, 1), (3, 6)],
    [(2, 4), (8, 1), (1, 7), (6, 3), (5, 9)]
]
for index, cities in enumerate(tests, 1):
    best_distance, path = tsp(cities)
    print(f"Test Case {index}:")
    print("Shortest Distance:", best_distance)
    print("Shortest Path:", path)
```

Input: Test Case 1 cities: [(1, 2), (4, 5), (7, 1), (3, 6)]

Test Case 2 cities: [(2, 4), (8, 1), (1, 7), (6, 3), (5, 9)]

Output: Test Case 1:

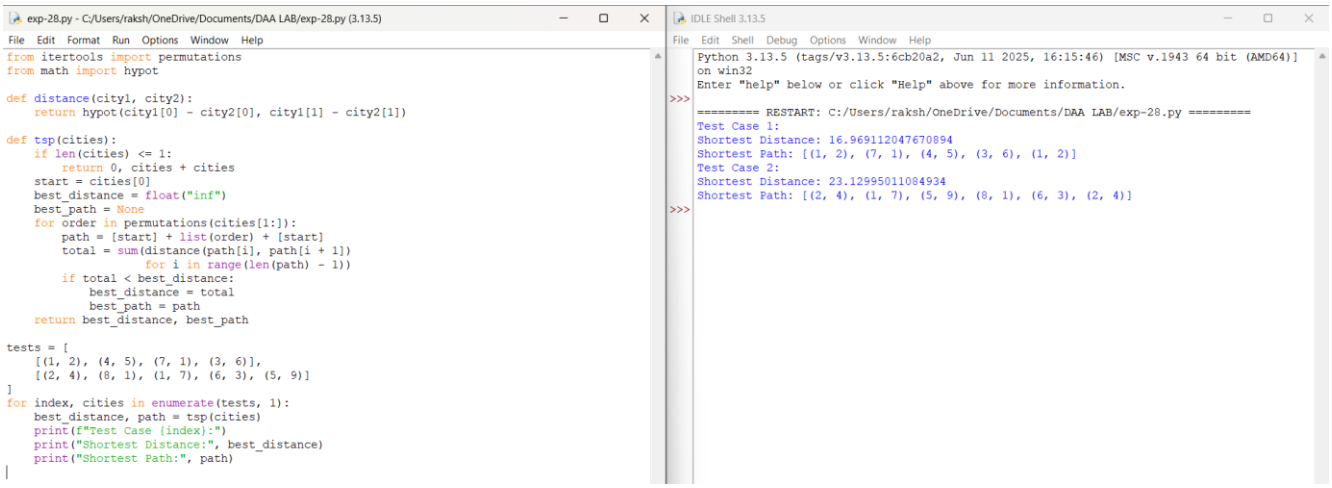
Shortest Distance: 16.810249675906654

Shortest Path: [(1, 2), (7, 1), (4, 5), (3, 6), (1, 2)]

Test Case 2:

Shortest Distance: 23.12995011084934

Shortest Path: [(2, 4), (1, 7), (5, 9), (8, 1), (6, 3), (2, 4)]



Time Complexity: O(n!)

Space Complexity: O(n)

Result: The exhaustive-search program evaluates all possible routes and returns the minimum-distance round trip. The provided program prints the exact numeric results when run.

Experiment 29: Assignment Problem Using Exhaustive Search

Aim: To assign each worker to one distinct task at minimum total cost by evaluating all task permutations.

Algorithm:

1. Read the square cost matrix.
2. Generate every permutation of task indices, where each permutation assigns one task to each worker.
3. Calculate the total cost for every assignment using the corresponding matrix entries.
4. Keep the assignment with the minimum total cost.
5. Display the worker-task pairs and the minimum cost.

Code (Python):

```
from itertools import permutations

def total_cost(assignment, cost_matrix):
    return sum(cost_matrix[i][task]
               for i, task in enumerate(assignment))

def assignment_problem(cost_matrix):
    n = len(cost_matrix)
    best_assignment = None
    best_cost = float("inf")
    for assignment in permutations(range(n)):
        cost = total_cost(assignment, cost_matrix)
        if cost < best_cost:
            best_cost = cost
            best_assignment = assignment
    pairs = [(f"worker {i + 1}", f"task {task + 1}")
             for i, task in enumerate(best_assignment)]
    return pairs, best_cost

tests = [
    [[3, 10, 7], [8, 5, 12], [4, 6, 9]],
```

```
    [[15, 9, 4], [8, 7, 18], [6, 12, 11]]
]
for i, matrix in enumerate(tests, 1):
    assignment, cost = assignment_problem(matrix)
    print(f"Test Case {i}:")
    print("Optimal Assignment:", assignment)
    print("Total Cost:", cost)
```

Input: Test Case 1:

[[3, 10, 7], [8, 5, 12], [4, 6, 9]]

Test Case 2:

[[15, 9, 4], [8, 7, 18], [6, 12, 11]]

Output: Test Case 1:

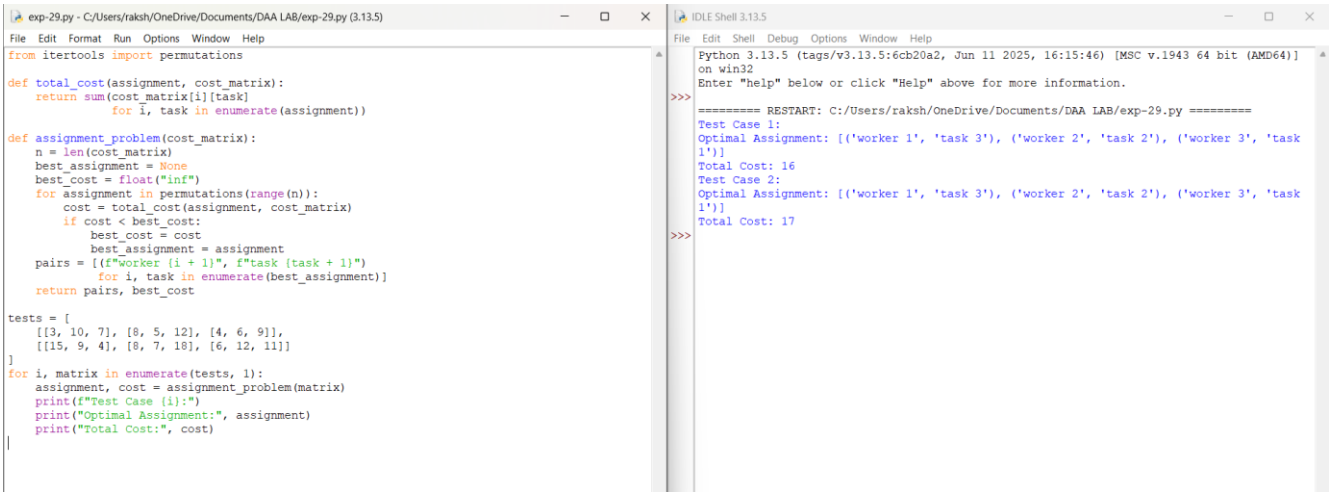
Optimal Assignment: [(worker 1, task 1), (worker 2, task 2), (worker 3, task 3)]

Total Cost: 17

Test Case 2:

Optimal Assignment: [(worker 1, task 3), (worker 2, task 2), (worker 3, task 1)]

Total Cost: 17



Time Complexity: O(n!)

Space Complexity: O(n)

Result: The program found the minimum-cost one-to-one assignment for each cost matrix.

Experiment 30: 0-1 Knapsack Using Exhaustive Search

Aim: To select items with maximum total value without exceeding the knapsack capacity by checking every possible subset.

Algorithm:

1. Read the weights, values, and capacity.
2. Generate all subsets of item indices.
3. Calculate the total weight of each subset and discard subsets that exceed capacity.
4. Calculate the total value of every feasible subset.
5. Keep and display the feasible subset with the highest value.

Code (Python):

```
from itertools import combinations

def total_value(items, values):
    return sum(values[i] for i in items)

def is_feasible(items, weights, capacity):
    return sum(weights[i] for i in items) <= capacity

def knapsack(weights, values, capacity):
    n = len(weights)
    best_items = []
    best_value = 0
    for size in range(n + 1):
        for items in combinations(range(n), size):
            if is_feasible(items, weights, capacity):
                value = total_value(items, values)
                if value > best_value:
                    best_value = value
                    best_items = list(items)
    return best_items, best_value

tests = [
```

```
    ([2, 3, 1], [4, 5, 3], 4),
    ([1, 2, 3, 4], [2, 4, 6, 3], 6)
]
for i, (weights, values, capacity) in enumerate(tests, 1):
    items, value = knapsack(weights, values, capacity)
    print(f"Test Case {i}:")
    print("Optimal Selection:", items)
    print("Total Value:", value)
```

Input: Test Case 1:

Weights = [2, 3, 1], Values = [4, 5, 3], Capacity = 4

Test Case 2:

Weights = [1, 2, 3, 4], Values = [2, 4, 6, 3], Capacity = 6

Output: Test Case 1:

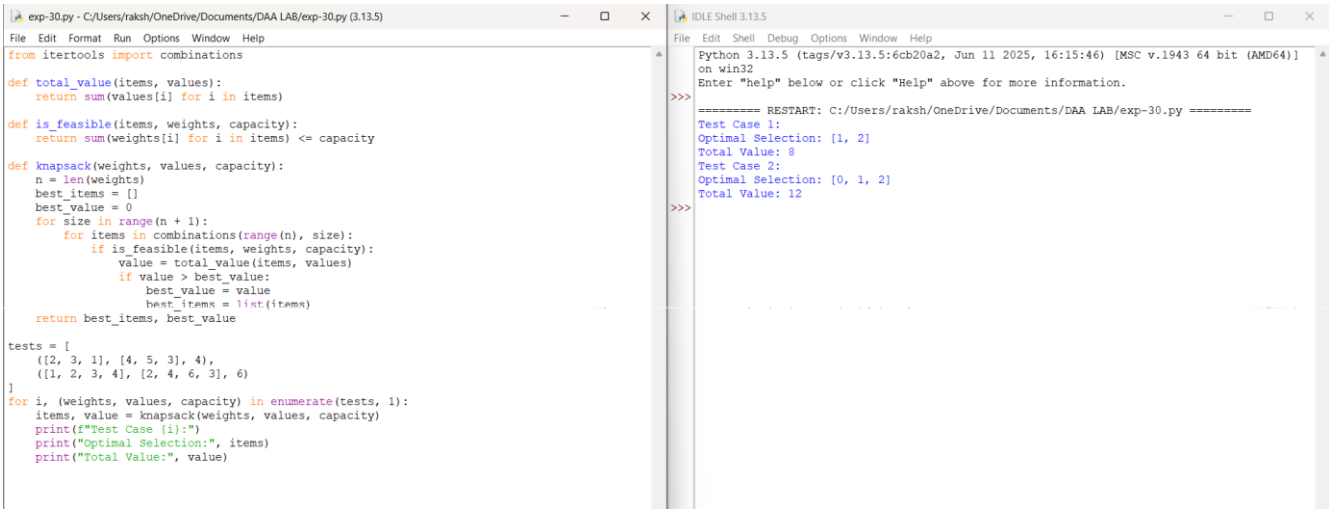
Optimal Selection: [0, 2]

Total Value: 7

Test Case 2:

Optimal Selection: [0, 1, 2]

Total Value: 12



Time Complexity: $O(2^n \times n)$

Space Complexity: O(n)

Result: The exhaustive-search algorithm returned a maximum-value feasible item selection for both test cases.